new idea

STORING DATA

Storing Data

Goal: store, maintain, search data

USPN: information on all students, teachers, grades, courses

Darty: products and all their information

Songs: artist, year of release, type of song, album cover

Banking: transactions with user accounts

Need to be able to:

update (add, delete, modify)

submit new grades of all students for the module `Databases`

search (called **querying**)

all teachers who taught C++ programming

Storing Data

First try: take pictures, screenshots of everything as image files on a computer



+ flexible, lots of information
- rigid, difficult to search/manipulate

Second try: a list of files (doc, pdf, xls, txt, ...) on your computer

+ easier to access data
- must write programs to search
- duplicate or inconsistent information
- multiple formats

Major Concerns: Scalability, Security, Efficiency, Concurrency, . . .

Storing Data

For wide use, **must** satisfy *many* criteria:

fast search

many concurrent users

consistency when modifying data

secure from unauthorized access

be able to recover from errors

always correct search reply

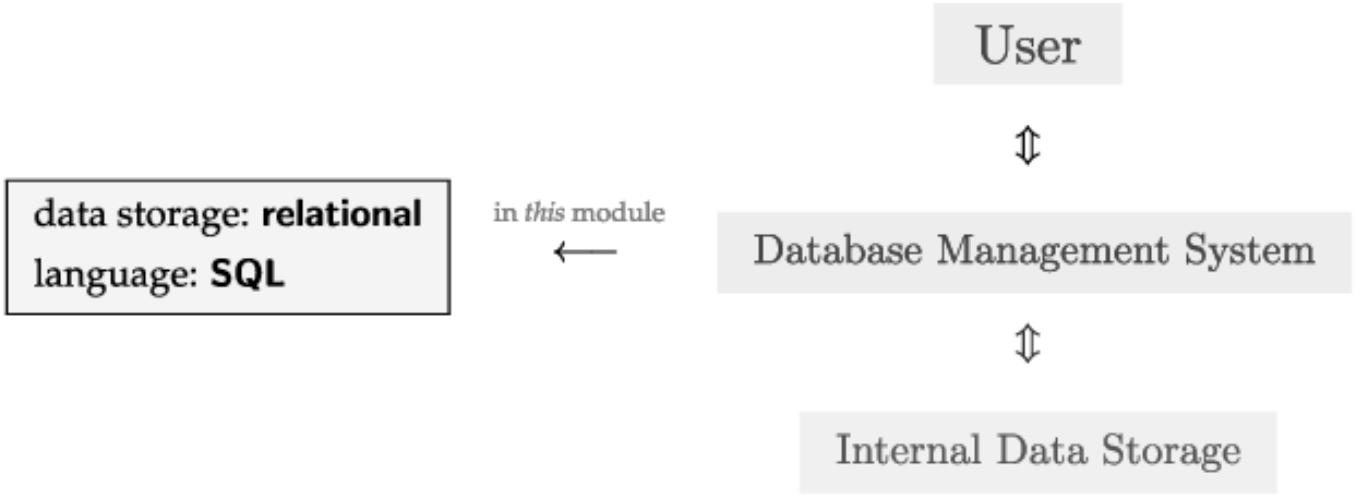
confidentiality

Databases

A database management system (DBMS) consists of:

data storage—called a **database**

language—to manipulate/search a database

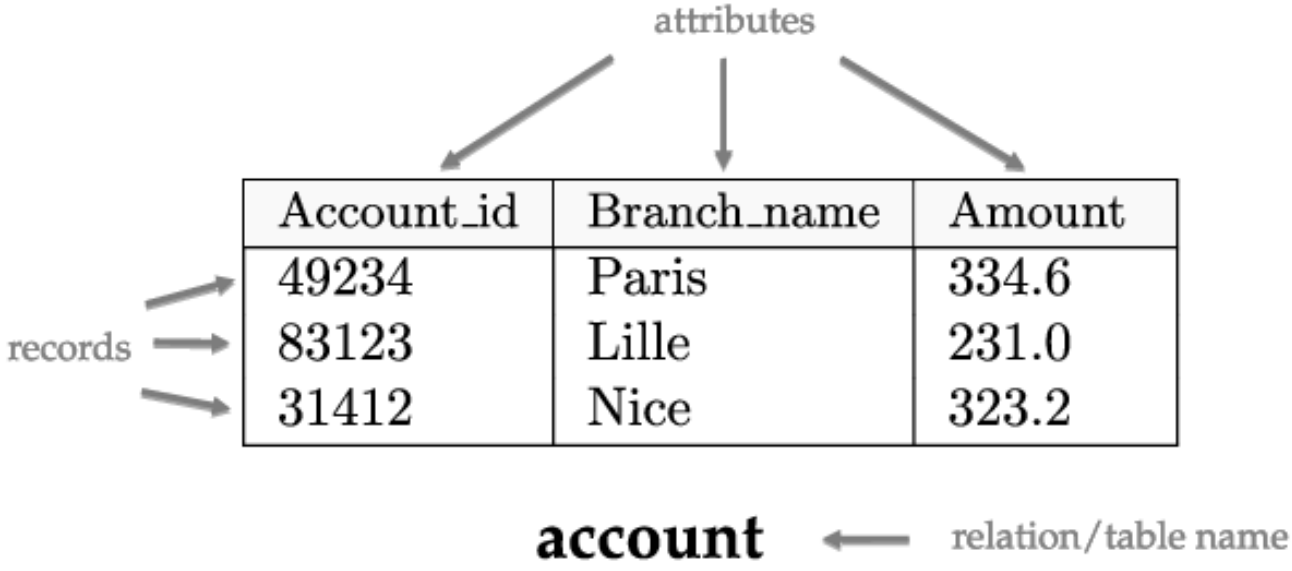


new idea

RELATIONAL DATABASE

Relations (Tables)

A relation is just a table of data

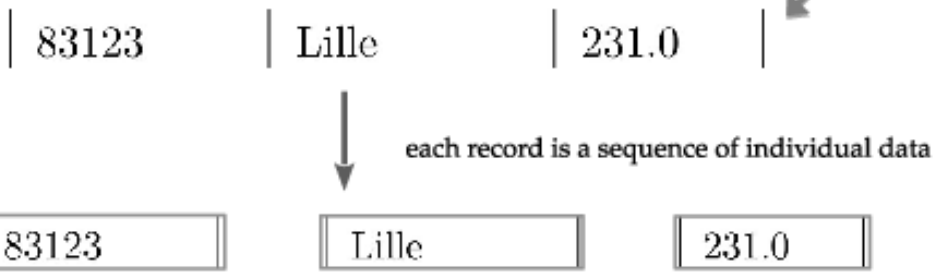


Relations (Tables)

Account_id	Branch_name	Amount
49234	Paris	334.6
83123	Lille	231.0
31412	Nice	323.2

account

each row in the table represents one record



Rows

Account_id	Branch_name	Amount
49234	Paris	334.6
83123	Lille	231.0
31412	Nice	323.2

account

record

each record is a *sequence* of individual data

(Account_id: 83123, Branch_name: Lille, Amount: 231.0)

or a shorter way:

(83123, Lille, 231.0)

mathematical name : a **tuple**

Columns

account

Account_id	Branch_name	Amount
49234	Paris	334.6
83123	Lille	231.0
31412	Nice	323.2

attribute

also called its **domain**

each attribute has a data type

Account_id	integer
Branch_name	string
Amount	numeric

null = value unknown or unspecified

technical phrase: 'domain is atomic'

indivisible units of data

handle it as a *single piece of data*

74242	Nice, Marseille	423.2

✗

Relations (Tables)

account

Account_id	Branch_name	Amount
49234	Paris	334.6
83123	Lille	231.0
31412	Nice	323.2

relation/table

there is no order among tuples

no duplicate tuples

(Account_id: 83123, Branch_name: Lille, Amount: 231.0)
(Account_id: 49234, Branch_name: Paris, Amount: 334.6)
(Account_id: 31412, Branch_name: Nice, Amount: 323.2)

same!

(Account_id: 31412, Branch_name: Nice, Amount: 323.2)
(Account_id: 83123, Branch_name: Lille, Amount: 231.0)
(Account_id: 49234, Branch_name: Paris, Amount: 334.6)

mathematical name :

a relation is a **set** of tuples

Relations (Tables)

To define a relation (table), we need to specify :

the list of attribute names
domain of each attribute
} called a schema

account

Account_id	Branch_name	Amount
49234	Paris	334.6
83123	Lille	231.0
31412	Nice	323.2

schema of the table:

attribute name

attribute domain

table name → account (Account_id integer, Branch_name string, Amount numeric)

a shorter way to write this: account (Account_id, Branch_name, Amount)

Examples

schema: account (Account_id integer, Branch_name string, Amount float)

Account_id	Branch_name	Amount
49234	Paris	334.6
83123	Lille	231.0
31412	Nice	323.2
34234	Marseille	423.12
62433	Tour	391
31412	Nice	323.2

Valid table or not ? NO

Examples

schema: account (Account_id integer, Branch_name string, Amount float)

Account_id	Branch_name	Amount
49234	Paris	334.6
83123	Lille	231.0
31412	Nice	323.2
34234	Marseille	423.12
62433	Tour	391
31412	Nice	271.1

Valid table or not ? YES

No two tuples can have same values for **all** attributes

But can have same values for **some** attributes

Examples

schema: account (Account_id integer, Branch_name string, Amount float)

Account_id	Branch_name	Amount
49234	Paris	334.6
83123	Lille	231.0
31412	Nice	323.2
34234	Marseille	423.12
62433	Tour	391
31412	Nce	323.2

Valid table or not ? YES

The domain for `Branch_name` can be any string

Examples

schema: account (Account_id integer, Branch_name string, Amount float)

Account_id	Branch_name	Amount
49234	Paris	334.6
83123	Lille	231.0
31412	Nice	323.2
34234	Marseille	423.12
62433	Tour	391
31412	Nce	323.2

?

=

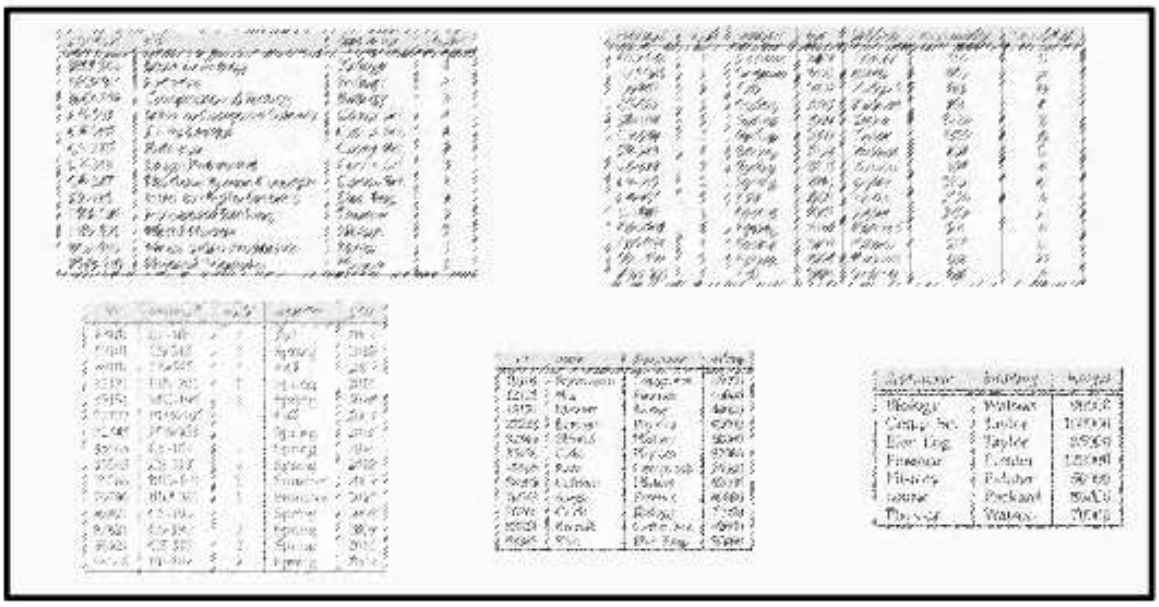
Account_id	Branch_name	Amount
49234	Paris	334.6
31412	Nce	323.2
31412	Nice	323.2
34234	Marseille	423.12
83123	Lille	231.0
62433	Tour	391

Are the two tables equal ? YES

order of rows does not matter!

Relational Database

A relational database consists of a set of tables



We will often use the University Database as an example in this module

teacher, student, course, section, teaches, takes, department

new idea

AN EXAMPLE

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

teacher

(from Database System Concepts)

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

course

(from Database System Concepts)

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

section

(from Database System Concepts)

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

teaches

(from Database System Concepts)

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

department

(from Database System Concepts)

id	*name*	*dept_name*	*tot_cred*
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

student

(from Database System Concepts)

id	course_id	sec_id	semester	year	grade
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

takes

(from Database System Concepts)

course

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

course (course_id, title, dept_name, credits)

section

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

section (course_id, sec_id, semester, year, building, room_number, time_slot_id)

teaches

id	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

teaches (id, course_id, sec_id, semester, year)

teacher

id	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
98345	Kim	Elec. Eng.	80000.00

teacher (id, name, dept_name, salary)

department

dept_name	building	budget
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

department (dept_name, building, budget)

Relational Database

A relational database consists of a set of tables











We will often use the University Database as an example in this module

teacher, student, course, section, teaches, takes, department

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

teacher

(from Database System Concepts)

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

course

(from Database System Concepts)

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

section

(from Database System Concepts)

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

teaches

(from Database System Concepts)

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

department

(from Database System Concepts)

id	*name*	*dept_name*	*tot_cred*
00128	Zhang	Comp. Sci.	102
12345	Shankar	Comp. Sci.	32
19991	Brandt	History	80
23121	Chavez	Finance	110
44553	Peltier	Physics	56
45678	Levy	Physics	46
54321	Williams	Comp. Sci.	54
55739	Sanchez	Music	38
70557	Snow	Physics	0
76543	Brown	Comp. Sci.	58
76653	Aoi	Elec. Eng.	60
98765	Bourikas	Elec. Eng.	98
98988	Tanaka	Biology	120

student

(from Database System Concepts)

id	*course_id*	*sec_id*	*semester*	*year*	*grade*
00128	CS-101	1	Fall	2009	A
00128	CS-347	1	Fall	2009	A-
12345	CS-101	1	Fall	2009	C
12345	CS-190	2	Spring	2009	A
12345	CS-315	1	Spring	2010	A
12345	CS-347	1	Fall	2009	A
19991	HIS-351	1	Spring	2010	B
23121	FIN-201	1	Spring	2010	C+
44553	PHY-101	1	Fall	2009	B-
45678	CS-101	1	Fall	2009	F
45678	CS-101	1	Spring	2010	B+
45678	CS-319	1	Spring	2010	B
54321	CS-101	1	Fall	2009	A-
54321	CS-190	2	Spring	2009	B+
55739	MU-199	1	Spring	2010	A-
76543	CS-101	1	Fall	2009	A
76543	CS-319	2	Spring	2010	A
76653	EE-181	1	Spring	2009	C
98765	CS-101	1	Fall	2009	C-
98765	CS-315	1	Spring	2010	B
98988	BIO-101	1	Summer	2009	A
98988	BIO-301	1	Summer	2010	

takes

(from Database System Concepts)

course

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems		
FIN-201	Investment Banking		
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

course (course_id, title, dept_name, credits)

section

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

section (course_id, sec_id, semester, year, building, room_number, time_slot_id)

teaches

id	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

teaches (ID, course_id, sec_id, semester, year)

teacher

id	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
	El Said	History	
	Gold	Physics	
45565	Katz	Comp. Sci.	
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
98345	Kim	Elec. Eng.	80000.00

teacher (ID, name, dept_name, salary)

department

dept_name	building	budget
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

department (dept_name, building, budget)

new idea

QUERIES

Using a Database

1. Design the database schema

2. Create all the tables

This is just to give you an idea

No need to understand this now

We will see details later!

for each table, enter the attribute names, and their domains

```
create table department (
  dept_name  varchar(20),
  building  varchar(15),
  budget numeric(12,2),
  primary key (dept_name)
 );
```

Using a Database

1. Design the database schema

2. Create all the tables

for each table, enter the attribute names, and their domains


```
create table department (
  dept_name  varchar(20),
  building  varchar(15),
  budget numeric(12,2),
  primary key (dept_name)
 );
```


many languages for queries

we will use **SQL**

- most common
- case-insensitive
- queries end with a semi-colon ;

3. Search information in the database called doing a query

The Rise of SQL › It's become the second programming language everyone needs to know



SQL dominated the jobs ranking in *IEEE Spectrum*'s interactive rankings of the top programming languages this year. Normally, the top position is occupied by Python or other mainstays, such as C, C++, Java, and JavaScript, but the sheer number of times employers said they wanted developers with SQL skills, albeit in addition to knowledge of a programming language, caused it to rise.

So what's behind SQL's year in the sun? The ever-increasing use of databases, for one. SQL has become the primary way by which to access and manage data stored in such databases—specifically relational databases, which represent data in table form with rows and columns. Databases serve as the foundation of many enterprise applications and are increasingly found in other places as well, for example taking the place of traditional file systems in smartphones.

The Top Programming Languages 2025 › Does AI mean the end for the Top Programming Languages?

BY STEPHEN CASS 23 SEP 2025 | 9 MIN READ |

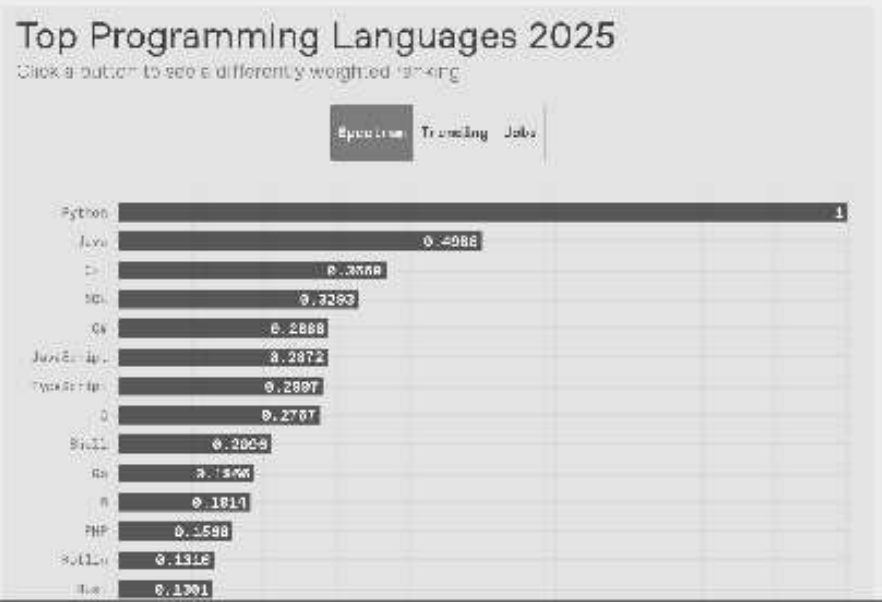
SHARE THIS STORY

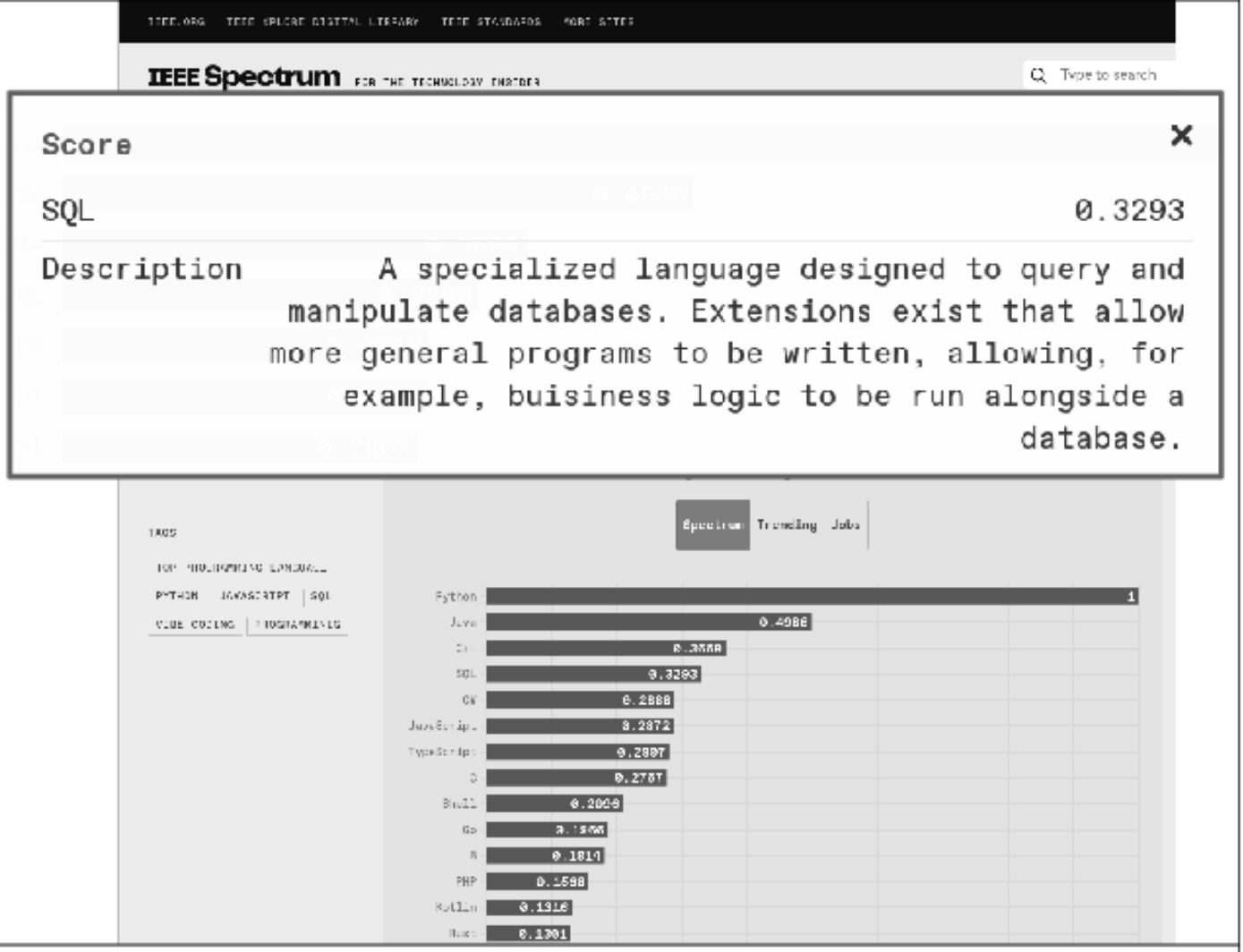
TAGS

TOP PROGRAMMING LANGUAGES

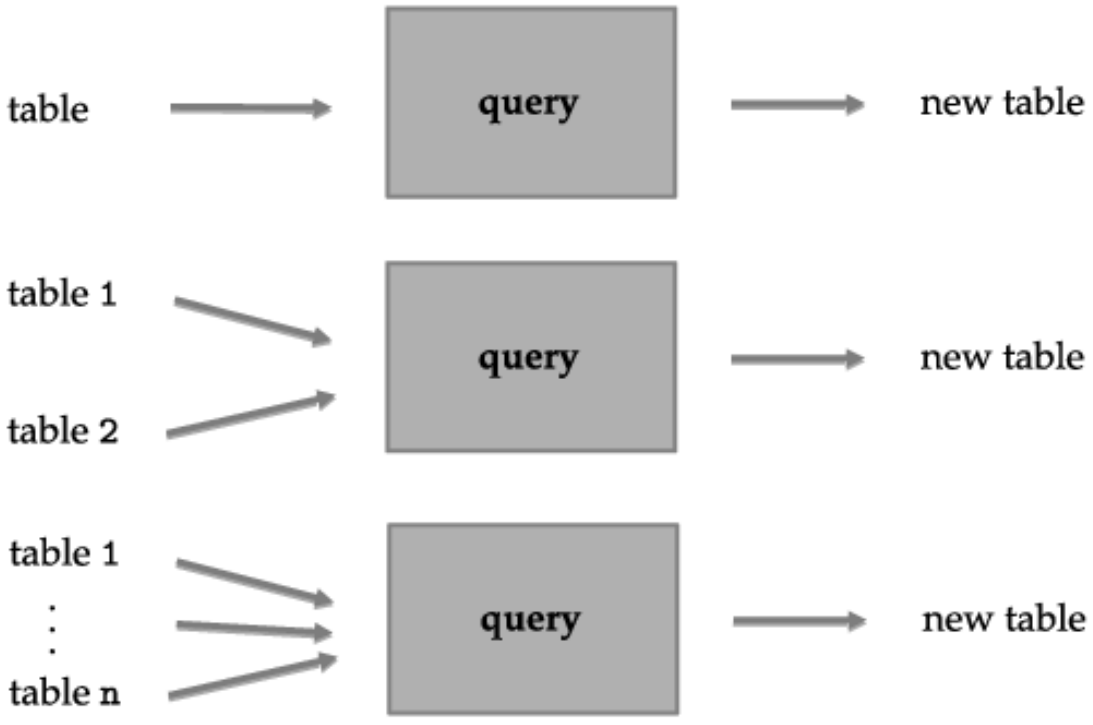
PYTHON JAVASCRIPT | SQL

VIBE CODING | PROGRAMMING





Database Queries



The result of a query is always **one table**

Database Queries

Each query does three things:

SQL keyword

1. **figure out the input table** → **FROM**
2. **choose output table rows** → **WHERE**
3. **choose output table columns** → **SELECT**

Query: Choose Rows

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Goal: **given a table: choose rows where credits ≤ 3**

→ **WHERE** **credits ≤ 3**

↓

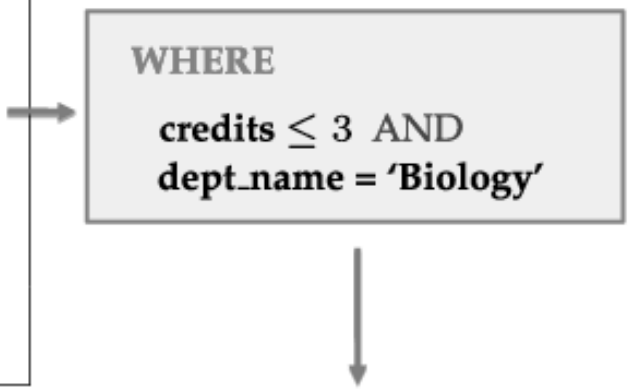
course_id	*title*	*dept_name*	*credits*
BIO-399	Computational Biology	Biology	3
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3

Query: Choose Rows

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

given a table: choose rows



course_id	*title*	*dept_name*	*credits*
BIO-399	Computational Biology	Biology	3

Query: Choose Columns

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

Goal: **given a table: choose columns**



title
Intro. to Biology
Genetics
Computational Biology
Intro. to Computer Science
Game Design
Robotics
Image Processing
Database System Concepts
Intro. to Digital Systems
Investment Banking
World History
Music Video Production
Physical Principles

new idea

SQL Syntax

Query: Choose Rows

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

input

choosing rows

FROM course

WHERE

credits ≤ 3

SELECT course_id, title, dept_name, credits

choosing columns

course_id	*title*	*dept_name*	*credits*
BIO-399	Computational Biology	Biology	3
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3

SQL syntax

```
SELECT course_id, title, dept_name, credits
FROM course
WHERE credits <= 3 ;
```

This order is obligatory: SELECT → FROM → WHERE

Query: Choose Rows

```
myuniversitydb=# SELECT course id, title, dept name, credits
                 FROM course
                 WHERE credits <= 3 ;
 course_id |           title            | dept_name  | credits
-----------+----------------------------+------------+---------
 BIO-399   | Computational Biology      | Biology    |       3
 CS-315    | Robotics                   | Comp. Sci. |       3
 CS-319    | Image Processing           | Comp. Sci. |       3
 CS-347    | Database System Concepts   | Comp. Sci. |       3
 EE-181    | Intro. to Digital Systems  | Elec. Eng. |       3
 FIN-201   | Investment Banking         | Finance    |       3
 HIS-351   | World History              | History    |       3
 MU-199    | Music Video Production     | Music      |       3
(8 rows)
```

Query: Choose Rows

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp.	
CS-190	Game Design	Comp.	
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

FROM course

WHERE

credits ≤ 3 AND
dept_name = 'Biology'

SELECT

SQL syntax

course_id	*title*	*dept_name*	*credits*
BIO-399	Computational Biology	Biology	3

```
SELECT course_id, title, dept_name, credits
FROM course
WHERE credits <= 3 AND dept_name = 'Biology' ;
```

Query: Choose Columns

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

given a table: choose columns

SELECT dept_name, credits

all

distinct

keywords

distinct

all

default behavior: all

keep duplicates

default case

dept_name	*credits*
Biology	4
Biology	4
Biology	3
Comp. Sci.	4
Comp. Sci.	4
Comp. Sci.	3
Comp. Sci.	3
Comp. Sci.	3
Elec. Eng.	3
Finance	3

SELECT all dept_name, credits

dept_name	*credits*
Biology	4
Biology	3
Comp. Sci.	4
Comp. Sci.	3
Elec. Eng.	3
Finance	3
History	3
Music	3
Physics	4

SELECT distinct dept_name, credits

Query: Choose Columns

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

dept_name	*credits*
Biology	4
Biology	3
Comp. Sci.	4
Comp. Sci.	3
Elec. Eng.	3
Finance	3
History	3
Music	3
Physics	4

```
SELECT distinct dept_name, credits
FROM course ;
```

```
SELECT all dept_name, credits
FROM course ;
```

this is the default ⇕ **same!**

```
SELECT dept_name, credits
FROM course ;
```

dept_name	*credits*
Biology	4
Biology	4
Biology	3
Comp. Sci.	4
Comp. Sci.	4
Comp. Sci.	3
Comp. Sci.	3
Comp. Sci.	3
Elec. Eng.	3
Finance	3
History	3
Music	3
Physics	4

Query: Choose Columns

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

adding WHERE is not obligatory

SQL syntax

```
SELECT title
FROM course ;
```

title
Intro. to Biology
Genetics
Computational Biology
Intro. to Computer Science
Game Design
Robotics
Image Processing
Database System Concepts
Intro. to Digital Systems
Investment Banking
World History
Music Video Production
Physical Principles

SELECT and FROM obligatory

```
FROM course;
```

```
myuniversitydb=# FROM course;
ERROR:  syntax error at or near "FROM"
LINE 1: FROM course;
        ^
```

```
FROM course
WHERE credits <= 3;
```

```
myuniversitydb=# FROM course
myuniversitydb-# WHERE credits <= 3;
ERROR:  syntax error at or near "FROM"
LINE 1: FROM course
        ^
```

```
SELECT
FROM course;
```

```
myuniversitydb=# SELECT
myuniversitydb-# FROM course;
--
(13 rows)
```

new idea

Product of Tables

Query: PRODUCT

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

product

department

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

course.course_id	*course.title*	*course.dept_name*	*course.credits*	*dpt.dept_name*	*dpt.building*	*dpt.budget*
BIO-101	Intro. to Biology	Biology	4	Biology	Watson	90000
BIO-101	Intro. to Biology	Biology	4	Comp. Sci.	Taylor	100000
BIO-101	Intro. to Biology	Biology	4	Elec. Eng.	Taylor	85000
BIO-101	Intro. to Biology	Biology	4	Finance	Painter	120000
BIO-101	Intro. to Biology	Biology	4	History	Painter	50000
BIO-101	Intro. to Biology	Biology	4	Music	Packard	80000
BIO-101	Intro. to Biology	Biology	4	Physics	Watson	70000
⋮						

Query: PRODUCT

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

13

product

department

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	T	85000.00
Finance	P	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

7

course.course_id	*course.title*	*course.dept_name*	*course.credits*	*dpt.dept_name*	*dpt.building*	*dpt.budget*
BIO-101	Intro. to Biology	Biology	4	Biology	Watson	90000
BIO-101	Intro. to Biology	Biology	4	Comp. Sci.	Taylor	100000
BIO-101	Intro. to Biology	Biology	4	Elec. Eng.	Taylor	85000
BIO-101	Intro. to Biology	Biology	4	Finance	Painter	120000
BIO-101	Intro. to Biology	Biology	4	History	Painter	50000
BIO-101	Intro. to Biology	Biology	4	Music	Packard	80000
BIO-101	Intro. to Biology	Biology	4	Physics	Watson	70000
BIO-301	Genetics	Biology	4	Biology	Watson	90000
BIO-301	Genetics	Biology	4	Comp. Sci.	Taylor	100000
BIO-301	Genetics	Biology	4	Elec. Eng.	Taylor	85000
BIO-301	Genetics	Biology	4	Finance	Painter	120000
BIO-301	Genetics	Biology	4	History	Painter	50000
BIO-301	Genetics	Biology	4	Music	Packard	80000
BIO-301	Genetics	Biology	4	Physics	Watson	70000

How many records in this new table ? $13 \times 7 = 91$

Use course.dept_name and department.dept_name to access attributes with same name !

$13 \times 7 = 91$ records in this product table



Queries: Basic Structure

① **FROM** table1, table2

the product table of table1 and table 2 is computed



to this product table, we apply the next step:

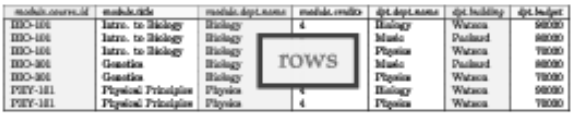


② **WHERE** condition

choose rows in the product table that satisfy condition

WHERE

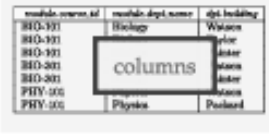
to these rows, we apply the final step:



③ **SELECT** attribute1, attribute2, attribute3

SELECT

for the rows, only output attribute 1, attribute 2, attribute 3 to get the final table



final table

new idea

Example Queries

Database Queries

teaches

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Compute the table of all teacher names and the courses they taught →

name	*course_id*
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
El Said	HIS-351
Katz	CS-101
Katz	CS-319
Crick	BIO-101
Crick	BIO-301
Brandt	CS-190
Brandt	CS-190
Brandt	CS-319
Kim	EE-181

information we need is in two tables

teacher name **is in teacher,** courses taught **is in teaches**

in SQL, we *cannot* access each table separately

Database Queries

teaches

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Compute the table of all teacher names and the courses they taught →

name	*course_id*
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
El Said	HIS-351
Katz	CS-101
Katz	CS-319
Crick	BIO-101
Crick	BIO-301
Brandt	CS-190
Brandt	CS-190
Brandt	CS-319
Kim	EE-181

1. take a product to get *all* pairs of rows from **teaches** and **teacher**
2. only keep rows where teaches.id and teacher.id match
3. return the teacher name and course_id of each remaining row

Database Queries

1. take a product to get all pairs of rows from **teaches** and **teacher**

FROM teaches, teacher

id	*name*	*dept_name*	*salary*	*id*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000.00	10101	CS-101	1	Fall	2009
12121	Wu	Finance	90000.00	10101	CS-101	1	Fall	2009
15151	Mozart	Music	40000.00	10101	CS-101	1	Fall	2009
22222	Einstein	Physics	95000.00	10101	CS-101	1	Fall	2009
32343	El Said	History	60000.00	10101	CS-101	1	Fall	2009
33456	Gold	Physics	87000.00	10101	CS-101	1	Fall	2009
45565	Katz	Comp. Sci.	75000.00	10101	CS-101	1	Fall	2009
58583	Califieri	History	62000.00	10101	CS-101	1	Fall	2009
76543	Singh	Finance	80000.00	10101	CS-101	1	Fall	2009
76766	Crick	Biology	72000.00	10101	CS-101	1	Fall	2009
83821	Brandt	Comp. Sci.	92000.00	10101	CS-101	1	Fall	2009
98345	Kim	Elec. Eng.	80000.00	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000.00	12121	FIN-201	1	Spring	2010
12121	Wu	Finance	90000.00	12121	FIN-201	1	Spring	2010
15151	Mozart	Music	40000.00	12121	FIN-201	1	Spring	2010
22222	Einstein	Physics	95000.00	12121	FIN-201	1	Spring	2010
32343	El Said	History	60000.00	12121	FIN-201	1	Spring	2010
33456	Gold	Physics	87000.00	12121	FIN-201	1	Spring	2010
45565	Katz	Comp. Sci.	75000.00	12121	FIN-201	1	Spring	2010
58583	Califieri	History	62000.00	12121	FIN-201	1	Spring	2010
76543	Singh	Finance	80000.00	12121	FIN-201	1	Spring	2010
76766	Crick	Biology	72000.00	12121	FIN-201	1	Spring	2010
83821	Brandt	Comp. Sci.	92000.00	12121	FIN-201	1	Spring	2010
98345	Kim	Elec. Eng.	80000.00	12121	FIN-201	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000.00	15151	MU-199	1	Spring	2010
12121	Wu	Finance	90000.00	15151	MU-199	1	Spring	2010

Database Queries

2. only keep rows where teaches.id and teacher.id match

id	*name*	*dept_name*	*salary*	*id*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000.00	10101	CS-101	1	Fall	2009
12121	Wu	Finance	90000.00	10101	CS-101	1	Fall	2009
15151	Mozart	Music	40000.00	10101	CS-101	1	Fall	2009
22222	Einstein	Physics	95000.00	10101	CS-101	1	Fall	2009
32343	El Said	History	60000.00	10101	CS-101	1	Fall	2009
33456	Gold	Physics	87000.00	10101	CS-101	1	Fall	2009
45565	Katz	Comp. Sci.	75000.00	10101	CS-101	1	Fall	2009
58583	Califieri	History	62000.00	10101	CS-101	1	Fall	2009
76543	Singh	Finance	80000.00	10101	CS-101	1	Fall	2009
76766	Crick	Biology	72000.00	10101	CS-101	1	Fall	2009
83821	Brandt	Comp. Sci.	92000.00	10101	CS-101	1	Fall	2009
98345	Kim	Elec. Eng.	80000.00	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000.00	12121	FIN-201	1	Spring	2010
12121	Wu	Finance	90000.00	12121	FIN-201	1	Spring	2010
15151	Mozart	Music	40000.00	12121	FIN-201	1	Spring	2010
22222	Einstein	Physics	95000.00	12121	FIN-201	1	Spring	2010
32343	El Said	History	60000.00	12121	FIN-201	1	Spring	2010
33456	Gold	Physics	87000.00	12121	FIN-201	1	Spring	2010
45565	Katz	Comp. Sci.	75000.00	12121	FIN-201	1	Spring	2010
58583	Califieri	History	62000.00	12121	FIN-201	1	Spring	2010
76543	Singh	Finance	80000.00	12121	FIN-201	1	Spring	2010
76766	Crick	Biology	72000.00	12121	FIN-201	1	Spring	2010
83821	Brandt	Comp. Sci.	92000.00	12121	FIN-201	1	Spring	2010
98345	Kim	Elec. Eng.	80000.00	12121	FIN-201	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000.00	15151	MU-199	1	Spring	2010
12121	Wu	Finance	90000.00	15151	MU-199	1	Spring	2010
15151	Mozart	Music	40000.00	15151	MU-199	1	Spring	2010

Database Queries

2. only keep rows where teaches.id and teacher.id match

WHERE teaches.id = teacher.id

id	*name*	*dept_name*	*salary*	*id*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000.00	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000.00	10101	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000.00	10101	CS-347	1	Fall	2009
12121	Wu	Finance	90000.00	12121	FIN-201	1	Spring	2010
15151	Mozart	Music	40000.00	15151	MU-199	1	Spring	2010
22222	Einstein	Physics	95000.00	22222	PHY-101	1	Fall	2009
32343	El Said	History	60000.00	32343	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000.00	45565	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000.00	45565	CS-319	1	Spring	2010
76766	Crick	Biology	72000.00	76766	BIO-101	1	Summer	2009
76766	Crick	Biology	72000.00	76766	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000.00	83821	CS-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000.00	83821	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000.00	83821	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000.00	98345	EE-181	1	Spring	2009

Database Queries

3. return the teacher name and course_id of each remaining row

SELECT name, course_id

id	*name*	*dept_name*	*salary*	*id*	*course_id*	*sec_id*	*semester*	*year*
10101	Srinivasan	Comp. Sci.	65000.00	10101	CS-101	1	Fall	2009
10101	Srinivasan	Comp. Sci.	65000.00	10101	CS-315	1	Spring	2010
10101	Srinivasan	Comp. Sci.	65000.00	10101	CS-347	1	Fall	2009
12121	Wu	Finance	90000.00	12121	FIN-201	1	Spring	2010
15151	Mozart	Music	40000.00	15151	MU-199	1	Spring	2010
22222	Einstein	Physics	95000.00	22222	PHY-101	1	Fall	2009
32343	El Said	History	60000.00	32343	HIS-351	1	Spring	2010
45565	Katz	Comp. Sci.	75000.00	45565	CS-101	1	Spring	2010
45565	Katz	Comp. Sci.	75000.00	45565	CS-319	1	Spring	2010
76766	Crick	Biology	72000.00	76766	BIO-101	1	Summer	2009
76766	Crick	Biology	72000.00	76766	BIO-301	1	Summer	2010
83821	Brandt	Comp. Sci.	92000.00	83821	CS-190	1	Spring	2009
83821	Brandt	Comp. Sci.	92000.00	83821	CS-190	2	Spring	2009
83821	Brandt	Comp. Sci.	92000.00	83821	CS-319	2	Spring	2010
98345	Kim	Elec. Eng.	80000.00	98345	EE-181	1	Spring	2009

Database Queries

teaches

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Use teacher.id and teaches.id to access attributes with same name !

Compute the table of all teacher names and the courses they taught

```
SELECT name, course_id
FROM teacher, teaches
WHERE teacher.id = teaches.id ;
```

name	*course_id*
Srinivasan	CS-101
Srinivasan	CS-315
Srinivasan	CS-347
Wu	FIN-201
Mozart	MU-199
Einstein	PHY-101
El Said	HIS-351
Katz	CS-101
Katz	CS-319
Crick	BIO-101
Crick	BIO-301
Brandt	CS-190
Brandt	CS-190
Brandt	CS-319
Kim	EE-181

PostgreSQL

```
myuniversitydb=# SELECT name, course_id
myuniversitydb-# FROM teacher, teaches
myuniversitydb-# WHERE teacher.id = teaches.id;
    name    | course_id
------------+-----------
 Srinivasan | CS-101
 Srinivasan | CS-315
 Srinivasan | CS-347
 Wu         | FIN-201
 Mozart     | MU-199
 Einstein   | PHY-101
 El Said    | HIS-351
 Katz       | CS-101
 Katz       | CS-319
 Crick      | BIO-101
 Crick      | BIO-301
 Brandt     | CS-190
 Brandt     | CS-190
 Brandt     | CS-319
 Kim        | EE-181
(15 rows)
```

Database Queries

teaches

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

```
SELECT id, name, course_id
FROM teacher, teaches
WHERE teacher.id = teaches.id ;
```



id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

teacher

```
myuniversitydb=# SELECT id, name, course_id
myuniversitydb-# FROM teacher, teaches
myuniversitydb-# WHERE teacher.id = teaches.id;
ERROR:  column reference "id" is ambiguous
LINE 1: SELECT id, name, course_id
                ^
```

SQL requires certain rules, even if not necessary logically!

Database Queries

The general framework of an SQL query:

③ output attributes of the chosen rows

SELECT $A_1, A_2, \ldots, A_n$

FROM $r_1, \ldots, r_m$

WHERE P

① construct table from $r_1 \times \cdots \times r_m$

② condition used to choose rows from $r_1 \times \cdots \times r_m$

IMPORTANT: syntax order is not the same as the logical order

Syntax

(how to write)

SELECT

FROM

WHERE

(how to think)

Logical Order of Processing

FROM → WHERE → SELECT

Relations

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

course (course_id, title, dept_name, credits)

section

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

section (course_id, sec_id, semester, year, building, room_number, time_slot_id)

teaches

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

teaches (ID, course_id, sec_id, semester, year)

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sc	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

teacher (ID, name, dept_name, salary)

department

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Music	Packard	80000.00
Physics	Watson	70000.00

department (dept_name, building, budget)

recall

SELECT

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the id, name and salary of all teachers

```
SELECT id, name, salary
FROM teacher;
```

SQL syntax

id	*name*	*salary*
10101	Srinivasan	65000.00
12121	Wu	90000.00
15151	Mozart	40000.00
22222	Einstein	95000.00
32343	El Said	60000.00
33456	Gold	87000.00
45565	Katz	75000.00
58583	Califieri	62000.00
76543	Singh	80000.00
76766	Crick	72000.00
83821	Brandt	92000.00
98345	Kim	80000.00

recall

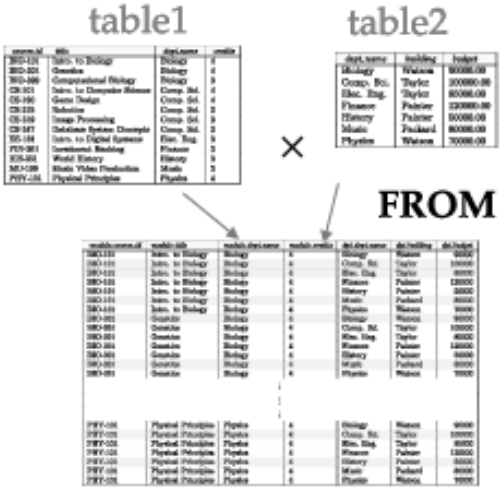
```
myuniversitydb=# SELECT id, name, salary
myuniversitydb-# FROM teacher;
  id   |    name    |  salary
-------+------------+----------
 10101 | Srinivasan | 65000.00
 12121 | Wu         | 90000.00
 15151 | Mozart     | 40000.00
 22222 | Einstein   | 95000.00
 32343 | El Said    | 60000.00
 33456 | Gold       | 87000.00
 45565 | Katz       | 75000.00
 58583 | Califieri  | 62000.00
 76543 | Singh      | 80000.00
 76766 | Crick      | 72000.00
 83821 | Brandt     | 92000.00
 98345 | Kim        | 80000.00
(12 rows)
```

recall

QUERIES: BASIC STRUCTURE

① **FROM** table1, table2

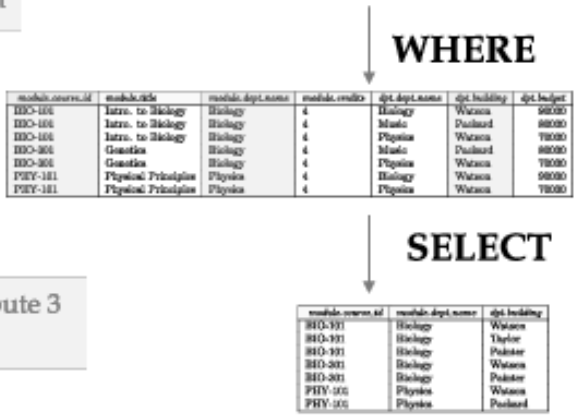
the product table of table1 and table 2 is computed



to this product table, we apply the next step:

② **WHERE** condition

choose rows in the product table that satisfy condition



to these rows, we apply the final step:

③ **SELECT** attribute1, attribute2, attribute3

for the rows, only output attribute 1, attribute 2, attribute 3 to get the final table

SELECT

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
myuniversitydb-# SELECT * FROM teacher;
  id   |     name     | dept_name  |  salary
-------+--------------+------------+----------
 10101 | Srinivasan   | Comp. Sci. | 65000.00
 12121 | Wu           | Finance    | 90000.00
 15151 | Mozart       | Music      | 40000.00
 22222 | Einstein     | Physics    | 95000.00
 32343 | El Said      | History    | 60000.00
 33456 | Gold         | Physics    | 87000.00
 45565 | Katz         | Comp. Sci. | 75000.00
 58583 | Califieri    | History    | 62000.00
 76543 | Singh        | Finance    | 80000.00
 76766 | Crick        | Biology    | 72000.00
 83821 | Brandt       | Comp. Sci. | 92000.00
 98345 | Kim          | Elec. Eng. | 80000.00
(12 rows)
```

all columns

```
SELECT * FROM teacher ;
```

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

new idea

LOGIC: AND
OR
NOT

AND

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT name, salary
FROM teacher
WHERE salary < 90000 AND salary > 70000;
```



name	*salary*
Gold	87000.00
Katz	75000.00
Singh	80000.00
Crick	72000.00
Kim	80000.00

```
myuniversitydb=# SELECT name, salary
myuniversitydb-# FROM teacher
myuniversitydb-# WHERE (salary < 90000) AND (salary > 70000);
 name  |  salary
-------+----------
 Gold  | 87000.00
 Katz  | 75000.00
 Singh | 80000.00
 Crick | 72000.00
 Kim   | 80000.00
(5 rows)
```

AND

true if *both* conditions are true

OR

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT name, salary
FROM teacher
WHERE salary > 90000 OR salary < 70000;
```



name	*salary*
Srinivasan	65000.00
Mozart	40000.00
Einstein	95000.00
El Said	60000.00
Califieri	62000.00
Brandt	92000.00

```
myuniversitydb=# SELECT name, salary
myuniversitydb-# FROM teacher
myuniversitydb-# WHERE salary > 90000 OR salary < 70000;
   name    |  salary
-----------+---------
 Srinivasan | 65000.00
 Mozart     | 40000.00
 Einstein   | 95000.00
 El Said    | 60000.00
 Califieri  | 62000.00
 Brandt     | 92000.00
(6 rows)
```

OR

true if *at least one* condition is true

NOT

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT name, salary
FROM teacher WHERE NOT (salary > 80000);
```



name	*salary*
Srinivasan	65000.00
Mozart	40000.00
El Said	60000.00
Katz	75000.00
Califieri	62000.00
Singh	80000.00
Crick	72000.00
Kim	80000.00

```
myuniversitydb=# SELECT name, salary
myuniversitydb-# FROM teacher
myuniversitydb-# WHERE NOT (salary > 80000);
    name    |  salary
------------+----------
 Srinivasan | 65000.00
 Mozart     | 40000.00
 El Said    | 60000.00
 Katz       | 75000.00
 Califieri  | 62000.00
 Singh      | 80000.00
 Crick      | 72000.00
 Kim        | 80000.00
(8 rows)
```

NOT

return the *opposite* evaluation

```
myuniversitydb=# SELECT NOT TRUE;
 ?column?
----------
 f
(1 row)
```

```
myuniversitydb=# SELECT NOT FALSE;
 ?column?
----------
 t
(1 row)
```

new idea

DISTINCT

DISTINCT

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT dept_name
FROM teacher ;
```

dept_name
Comp. Sci.
Finance
Music
Physics
History
Physics
Comp. Sci.
History
Finance
Biology
Comp. Sci.
Elec. Eng.

the order is not the same
order does not matter

```
SELECT distinct dept_name
FROM teacher ;
```

dept_name
Finance
History
Physics
Music
Comp. Sci.
Biology
Elec. Eng.

distinct is applied at the end
imagine doing the query without it, and then simply apply to final table

DISTINCT

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT distinct dept_name
FROM teacher ;
```

dept_name
Finance
History
Physics
Music
Comp. Sci.
Biology
Elec. Eng.

```
SELECT id, distinct dept_name
FROM teacher ;
```

```
myuniversitydb=# SELECT id, distinct dept_name
myuniversitydb-# FROM teacher;
ERROR:  syntax error at or near "distinct"
LINE 1: SELECT id, distinct dept_name
                   ^
```

```
SELECT distinct dept_name, id
FROM teacher ;
```

```
myuniversitydb=# SELECT distinct dept_name, id
myuniversitydb-# FROM teacher;
 dept_name  |  id
            +
 Finance    | 12121
 History    | 58583
 History    | 32343
 Physics    | 33456
 Finance    | 76543
 Comp. Sci. | 10101
 Biology    | 76766
 Music      | 15151
 Comp. Sci. | 83821
 Comp. Sci. | 45565
 Elec. Eng. | 98345
 Physics    | 22222
(12 rows)
```

distinct is applied to all columns!

so the full tuples are distinct, not individual columns

Logical Order of Processing

FROM ⟶ WHERE ⟶ SELECT



DISTINCT ?

FROM ⟶ WHERE ⟶ SELECT ⟶ DISTINCT

new idea

PRODUCTS

PRODUCTS

course

course_id	*title*	*dept_name*	*credits*
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-101	Intro. to Computer Science	Comp. Sci.	4
CS-190	Game Design	Comp. Sci.	4
CS-315	Robotics	Comp. Sci.	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

13 rows

department

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	T[illegible]	[illegible]000.00
Finance	P[illegible]	[illegible]000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

7 rows

```
SELECT *
FROM course, department ;
```

course_id	*title*	*dept_name*	*credits*	*dept_name*	*building*	*budget*
BIO-101	Intro. to Biology	Biology	4	Biology	Watson	90000.00
BIO-301	Genetics	Biology	4	Biology	Watson	90000.00
BIO-399	Computational Biology	Biology	3	Biology	Watson	90000.00
CS-101	Intro. to Computer Science	Comp. Sci.	4	Biology	Watson	90000.00
CS-190	Game Design	Comp. Sci.	4	Biology	Watson	90000.00
CS-315	Robotics	C[illegible]			Watson	90000.00
CS-319	Image Processing	C[illegible]			Watson	90000.00
CS-347	Database System Concepts	Comp. Sci.	3	Biology	Watson	90000.00
EE-181	Intro. to Digital Systems	Elec. Eng.	3	Biology	Watson	90000.00
FIN-201	Investment Banking	Finance	3	Biology	Watson	90000.00
HIS-351	World History	History	3	Biology	Watson	90000.00
MU-199	Music Video Production	Music	3	Biology	Watson	90000.00
PHY-101	Physical Principles	Physics	4	Biology	Watson	90000.00
BIO-101	Intro. to Biology	Biology	4	Comp. Sci.	Taylor	100000.00
BIO-301	Genetics	Biology	4	Comp. Sci.	Taylor	100000.00
BIO-399	Computational Biology	Biology	3	Comp. Sci.	Taylor	100000.00
CS-101	Intro. to Computer Science	Comp. Sci.	4	Comp. Sci.	Taylor	100000.00
CS-190	Game Design	Comp. Sci.	4	Comp. Sci.	Taylor	100000.00
CS-315	Robotics	Comp. Sci.	3	Comp. Sci.	Taylor	100000.00
CS-319	Image Processing	Comp. Sci.	3	Comp. Sci.	Taylor	100000.00
CS-347	Database System Concepts	Comp. Sci.	3	Comp. Sci.	Taylor	100000.00
EE-181	Intro. to Digital Systems	Elec. Eng.	3	Comp. Sci.	Taylor	100000.00
FIN-201	Investment Banking	Finance	3	Comp. Sci.	Taylor	100000.00
HIS-351	World History	History	3	Comp. Sci.	Taylor	100000.00
MU-199	Music Video Production	Music	3	Comp. Sci.	Taylor	100000.00
PHY-101	Physical Principles	Physics	4	Comp. Sci.	Taylor	100000.00
BIO-101	Intro. to Biology	Biology	4	Elec. Eng.	Taylor	85000.00
BIO-301	Genetics	Biology	4	Elec. Eng.	Taylor	85000.00
BIO-399	Computational Biology	Biology	3	Elec. Eng.	Taylor	85000.00
CS-101	Intro. to Computer Science	Comp. Sci.	4	Elec. Eng.	Taylor	85000.00
CS-190	Game Design	Comp. Sci.	4	Elec. Eng.	Taylor	85000.00
CS-315	Robotics	Comp. Sci.	3	Elec. Eng.	Taylor	85000.00
CS-319	Image Processing	Comp. Sci.	3	Elec. Eng.	Taylor	85000.00
CS-347	Database System Concepts	Comp. Sci.	3	Elec. Eng.	Taylor	85000.00
EE-181	Intro. to Digital Systems	Elec. Eng.	3	Elec. Eng.	Taylor	85000.00
FIN-201	Investment Banking	Finance	3	Elec. Eng.	Taylor	85000.00
HIS-351	World History	History	3	Elec. Eng.	Taylor	85000.00
MU-199	Music Video Production	Music	3	Elec. Eng.	Taylor	85000.00
PHY-101	Physical Principles	Physics	4	Elec. Eng.	Taylor	85000.00
BIO-101	Intro. to Biology	Biology	4	Finance	Painter	120000.00
BIO-301	Genetics	Biology	4	Finance	Painter	120000.00
BIO-399	Computational Biology	Biology	3	Finance	Painter	120000.00
CS-101	Intro. to Computer Science	Comp. Sci.	4	Finance	Painter	120000.00
CS-190	Game Design	Comp. Sci.	4	Finance	Painter	120000.00
CS-315	Robotics	Comp. Sci.	3	Finance	Painter	120000.00
CS-319	Image Processing	Comp. Sci.	3	Finance	Painter	120000.00
CS-347	Database System Concepts	Comp. Sci.	3	Finance	Painter	120000.00
EE-181	Intro. to Digital Systems	Elec. Eng.	3	Finance	Painter	120000.00
FIN-201	Investment Banking	Finance	3	Finance	Painter	120000.00
HIS-351	World History	History	3	Finance	Painter	120000.00
MU-199	Music Video Production	Music	3	Finance	Painter	120000.00
PHY-101	Physical Principles	Physics	4	Finance	Painter	120000.00
BIO-101	Intro. to Biology	Biology	4	History	Painter	50000.00
BIO-301	Genetics	Biology	4	History	Painter	50000.00
BIO-399	Computational Biology	Biology	3	History	Painter	50000.00
CS-101	Intro. to Computer Science	Comp. Sci.	4	History	Painter	50000.00
CS-190	Game Design	Comp. Sci.	4	History	Painter	50000.00
CS-315	Robotics	Comp. Sci.	3	History	Painter	50000.00
CS-319	Image Processing	Comp. Sci.	3	History	Painter	50000.00
CS-347	Database System Concepts	Comp. Sci.	3	History	Painter	50000.00
EE-181	Intro. to Digital Systems	Elec. Eng.	3	History	Painter	50000.00
FIN-201	Investment Banking	Finance	3	History	Painter	50000.00
HIS-351	World History	History	3	History	Painter	50000.00
MU-199	Music Video Production	Music	3	History	Painter	50000.00
PHY-101	Physical Principles	Physics	4	History	Painter	50000.00
BIO-101	Intro. to Biology	Biology	4	Music	Packard	80000.00
BIO-301	Genetics	Biology	4	Music	Packard	80000.00
BIO-399	Computational Biology	Biology	3	Music	Packard	80000.00
CS-101	Intro. to Computer Science	Comp. Sci.	4	Music	Packard	80000.00
CS-190	Game Design	Comp. Sci.	4	Music	Packard	80000.00
CS-315	Robotics	Comp. Sci.	3	Music	Packard	80000.00
CS-319	Image Processing	Comp. Sci.	3	Music	Packard	80000.00
CS-347	Database System Concepts	Comp. Sci.	3	Music	Packard	80000.00
EE-181	Intro. to Digital Systems	Elec. Eng.	3	Music	Packard	80000.00
FIN-201	Investment Banking	Finance	3	Music	Packard	80000.00
HIS-351	World History	History	3	Music	Packard	80000.00
MU-199	Music Video Production	Music	3	Music	Packard	80000.00
PHY-101	Physical Principles	Physics	4	Music	Packard	80000.00
BIO-101	Intro. to Biology	Biology	4	Physics	Watson	70000.00
BIO-301	Genetics	Biology	4	Physics	Watson	70000.00
BIO-399	Computational Biology	Biology	3	Physics	Watson	70000.00
CS-101	Intro. to Computer Science	Comp. Sci.	4	Physics	Watson	70000.00
CS-190	Game Design	Comp. Sci.	4	Physics	Watson	70000.00
CS-315	Robotics	Comp. Sci.	3	Physics	Watson	70000.00
CS-319	Image Processing	Comp. Sci.	3	Physics	Watson	70000.00
CS-347	Database System Concepts	Comp. Sci.	3	Physics	Watson	70000.00
EE-181	Intro. to Digital Systems	Elec. Eng.	3	Physics	Watson	70000.00
FIN-201	Investment Banking	Finance	3	Physics	Watson	70000.00
HIS-351	World History	History	3	Physics	Watson	70000.00
MU-199	Music Video Production	Music	3	Physics	Watson	70000.00
PHY-101	Physical Principles	Physics	4	Physics	Watson	70000.00

$13 \times 7 = 91$ rows

$13 \times 7 = 91$ rows



PRODUCTS

table2

table2_id	*table2_name*
10	tab2row1
20	tab2row2

table1

table1_id	*table1_name*
1	tab1row1
2	tab1row2
3	tab1row3

SELECT *
FROM table1, table2, table3 ;

table3

table3_id	*table3_name*
100	tab3row1
200	tab3row2

table1_id	*table1_name*	*table2_id*	*table2_name*	*table3_id*	*table3_name*
1	tab1row1	10	tab2row1	100	tab3row1
1	tab1row1	10	tab2row1	200	tab3row2
1	tab1row1	20	tab2row2	100	tab3row1
1	tab1row1	20	tab2row2	200	tab3row2
2	tab1row2	10	tab2row1	100	tab3row1
2	tab1row2	10	tab2row1	200	tab3row2
2	tab1row2	20	tab2row2	100	tab3row1
2	tab1row2	20	tab2row2	200	tab3row2
3	tab1row3	10	tab2row1	100	tab3row1
3	tab1row3	10	tab2row1	200	tab3row2
3	tab1row3	20	tab2row2	100	tab3row1
3	tab1row3	20	tab2row2	200	tab3row2

$3 \times 2 \times 2 = 12$ rows

PRODUCTS

department

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

department

dept_name	*building*	*budget*
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

```
SELECT *
FROM department, department;
```

```
myuniversitydb=# SELECT *
myuniversitydb-# FROM department, department;
ERROR:  table name "department" specified more than once
```

How to do a product of a table with itself ?

Products with Renaming

department

dept_name	building	budget
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

corelation name or table alias

```
SELECT * FROM department AS A, department AS B;
```

dept_name	building	budget
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00
History	Painter	50000.00
Music	Packard	80000.00
Physics	Watson	70000.00

department

dept_name	building	budget	dept_name	building	budget
Biology	Watson	90000.00	Biology	Watson	90000.00
Biology	Watson	90000.00	Comp. Sci.	Taylor	100000.00
Biology	Watson	90000.00	Elec. Eng.	Taylor	85000.00
Biology	Watson	90000.00	Finance	Painter	120000.00
Biology	Watson	90000.00	History	Painter	50000.00
Biology	Watson	90000.00	Music	Packard	80000.00
Biology	Watson	90000.00	Physics	Watson	70000.00
Comp. Sci.	Taylor	100000.00	Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00	Comp. Sci.	Taylor	100000.00
		100000.00	Elec. Eng.	Taylor	85000.00
		100000.00	Finance	Painter	120000.00
		100000.00	History	Painter	50000.00
		100000.00	Music	Packard	80000.00
		100000.00	Physics	Watson	70000.00
Elec. Eng.	Taylor	85000.00	Biology	Watson	90000.00
Elec. Eng.	Taylor	85000.00	Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00	Elec. Eng.	Taylor	85000.00
Elec. Eng.	Taylor	85000.00	Finance	Painter	120000.00
Elec. Eng.	Taylor	85000.00	History	Painter	50000.00
Elec. Eng.	Taylor	85000.00	Music	Packard	80000.00
Elec. Eng.	Taylor	85000.00	Physics	Watson	70000.00
Finance	Painter	120000.00	Biology	Watson	90000.00
Finance	Painter	120000.00	Comp. Sci.	Taylor	100000.00
Finance	Painter	120000.00	Elec. Eng.	Taylor	85000.00
Finance	Painter	120000.00	Finance	Painter	120000.00
Finance	Painter	120000.00	History	Painter	50000.00
Finance	Painter	120000.00	Music	Packard	80000.00
Finance	Painter	120000.00	Physics	Watson	70000.00
History	Painter	50000.00	Biology	Watson	90000.00
History	Painter	50000.00	Comp. Sci.	Taylor	100000.00
History	Painter	50000.00	Elec. Eng.	Taylor	85000.00
History	Painter	50000.00	Finance	Painter	120000.00
History	Painter	50000.00	History	Painter	50000.00
History	Painter	50000.00	Music	Packard	80000.00
History	Painter	50000.00	Physics	Watson	70000.00
Music	Packard	80000.00	Biology	Watson	90000.00

Products

teacher

id	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the names of all teachers whose salary is greater than at least one teacher in the Computer Science department.

```
SELECT A.name
FROM teacher AS A, teacher AS B
WHERE A.salary > B.salary
        AND B.dept_name = 'Comp. Sci.';
```

```
myuniversitydb=# SELECT A.name
myuniversitydb-# FROM teacher AS A, teacher as B
myuniversitydb-# WHERE A.salary > B.salary
myuniversitydb #   AND B.dept_name = 'Comp. Sci.';
   name
----------
 Wu
 Wu
 Einstein
 Einstein
 Einstein
 Gold
 Gold
 Katz
 Singh
 Singh
 Crick
 Brandt
 Brandt
 Kim
 Kim
(15 rows)
```

name
Wu
Wu
Einstein
Einstein
Einstein
Gold
Gold
Katz
Singh
Singh
Crick
Brandt
Brandt
Kim
Kim

PRODUCTS

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the names of all teachers whose salary is greater than at least one teacher in the Computer Science department.

```
SELECT distinct A.name
FROM teacher AS A, teacher AS B
WHERE A.salary > B.salary
      AND B.dept_name = 'Comp. Sci.';
```

SELECT ~~distinct~~ A.name
FROM teacher **AS** A, teacher **AS** B
WHERE A.salary > B.salary
and B.dept_name = 'Comp. Sci.'

→

name
Wu
Wu
Einstein
Einstein
Einstein
Gold
Gold
Katz
Singh
Singh
Crick
Brandt
Brandt
Kim
Kim

distinct →

name
Einstein
Katz
Singh
Kim
Brandt
Wu
Crick
Gold

SELECT distinct A.name
FROM teacher **AS** A, teacher **AS** B
WHERE A.salary > B.salary
and B.dept_name = 'Comp. Sci.'

PRODUCTS

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

teaches

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

can use `table.*` to select all attributes of the table

```
SELECT teacher.*
FROM teacher, teaches
WHERE teacher.id = teaches.id;
```

↓

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
10101	Srinivasan	Comp. Sci.	65000.00
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
45565	Katz	Comp. Sci.	75000.00
45565	Katz	Comp. Sci.	75000.00
76766	Crick	Biology	72000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
83821	Brandt	Comp. Sci.	92000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

new idea

AGGREGATIONS

Aggregate Functions

Aggregate functions: take a set of values, and return a ***single*** value

avg **sum** **max** **min** **count**

input must be numbers

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT avg(salary)
FROM teacher;
```

avg
74833.333333333333

```
SELECT avg(salary)
FROM teacher
WHERE dept_name = 'Comp. Sci.';
```

$$\frac{65000 + 75000 + 92000}{3} =$$

avg
77333.333333333333

Aggregate Functions

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT avg(salary)
FROM teacher;
```

① **SELECT** salary
FROM teacher

salary
65000.00
90000.00
40000.00
95000.00
60000.00
87000.00
75000.00
62000.00
80000.00
72000.00
92000.00
80000.00

② take avg

avg
74833.333333333333

this works correctly because we kept duplicates in query!

Aggregate Functions

teacher avg sum max min count

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Important: aggregate functions **always** collapse output to **ONE** row

```
SELECT salary
FROM teacher;
```

salary
65000.00
90000.00
40000.00
95000.00
60000.00
87000.00
75000.00
62000.00
80000.00
72000.00
92000.00
80000.00

```
SELECT max(salary)
FROM teacher;
```

```
myuniversitydb=# SELECT max(salary) FROM teacher;
   max
----------
 95000.00
(1 row)
```

```
SELECT count(*)
FROM teacher;
```

⇕ same!

```
SELECT min(salary)
FROM teacher;
```

```
myuniversitydb # SELECT min(salary) FROM teacher;
   min
---------
 40000.00
(1 row)
```

```
SELECT sum(salary)
FROM teacher;
```

```
myuniversitydb=# SELECT sum(salary) FROM teacher;
   sum
----------
 898000.00
(1 row)
```

```
SELECT count(salary)
FROM teacher;
```

```
myuniversitydb=# SELECT count(salary) FROM teacher;
 count
-------
    12
(1 row)
```

Aggregate Functions

teacher avg sum max min count

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT count(dept_name)
FROM teacher;
```

count
12

```
SELECT count(distinct dept_name)
FROM teacher;
```

count
7

```
SELECT sum(salary)
FROM teacher;
```

sum
898000.00

```
SELECT sum(distinct salary)
FROM teacher;
```

sum
818000.00

Aggregate Functions

teaches

id	*course_id*	*sec_id*	*semester*	*year*
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
32343	HIS-351	1	Spring	2010
45565	CS-101	1	Spring	2010
45565	CS-319	1	Spring	2010
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

Find the total number of teachers who taught a course in Spring 2010 semester.

```
SELECT count (distinct id)
FROM teaches
WHERE semester='Spring' AND year=2010;
```

count
6

```
SELECT id
FROM teaches
WHERE semester = 'Spring' and year = 2010
```

→ **SELECT** distinct id → **SELECT** count(distinct id)

id
10101
12121
15151
32343
45565
45565
83821

id
10101
12121
15151
32343
45565
83821

count
6

Logical Order of Processing

FROM ⟶ WHERE ⟶ SELECT

↓ DISTINCT ?

FROM ⟶ WHERE ⟶ SELECT ⟶ DISTINCT

↓ AGGREGATIONS ?

FROM ⟶ WHERE ⟶ AGGREGATIONS ⟶ SELECT ⟶ DISTINCT
⟶ aggregations available from here onwards

Aggregate Functions

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT sum(salary)
FROM teacher;
```

→

sum
898000.00

aggregate functions collapse to **ONE** row
but what id to put in that row ?
problem: many different id values!

```
SELECT max(id), sum(salary)
FROM teacher;
```

↓

max	*sum*
98345	898000.00

```
SELECT id, sum(salary)
FROM teacher
```

↓

```
myuniversitydb=# SELECT id, sum(salary) FROM teacher;
ERROR:  column "teacher.id" must appear in the GROUP BY clause or be used in an aggregate function
LINE 1: SELECT id, sum(salary) FROM teacher;
               ^
```

Aggregate Functions

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

cannot use aggregate functions in a **WHERE** clause

WHERE condition looks only at data *row by row*

in fact, **avg** calculated over the rows which *pass* the **WHERE** condition

FROM ⟶ WHERE ⟶ AGGREGATIONS ⟶ SELECT ⟶ DISTINCT

```
SELECT name, salary
FROM teacher
WHERE salary > avg(salary);
```

```
myuniversitydb=# SELECT name, salary
myuniversitydb-# FROM teacher
myuniversitydb-# WHERE salary > avg(salary);
ERROR:  aggregate functions are not allowed in WHERE
LINE 3: WHERE salary > avg(salary);
                       ^
```

Important:

in SQL, **don't go by intuition**.

Things that seem "natural" don't always work.

It is essential to be attentive and truly understand the logical processing behind the syntax.


```
SELECT name, salary
FROM teacher
WHERE salary > avg(salary);
```

```
myuniversitydb=# SELECT name, salary
myuniversitydb-# FROM teacher
myuniversitydb-# WHERE salary > avg(salary);
ERROR:  aggregate functions are not allowed in WHERE
LINE 3: WHERE salary > avg(salary);
                       ^
```

Logical Order of Processing

FROM ⟶ WHERE ⟶ SELECT

↓ **DISTINCT ?**

FROM ⟶ WHERE ⟶ SELECT ⟶ DISTINCT

↓ **AGGREGATIONS ?**

FROM ⟶ WHERE ⟶ AGGREGATIONS ⟶ SELECT ⟶ DISTINCT
⟶ aggregations available from here onwards

Good Naming Practices

Good coding style in SQL :

tables, attribute names in lowercase letters

write all SQL keywords in UPPERCASE

follow indentation consistently

each line has at most 80 characters

new idea

GROUP BY

RELATIONS

course

course_id	title	dept_name	credits
BIO-101	Intro. to Biology	Biology	4
BIO-301	Genetics	Biology	4
BIO-399	Computational Biology	Biology	3
CS-319	Image Processing	Comp. Sci.	3
CS-347	Database System Concepts	Comp. Sci.	3
EE-181	Intro. to Digital Systems	Elec. Eng.	3
FIN-201	Investment Banking	Finance	3
HIS-351	World History	History	3
MU-199	Music Video Production	Music	3
PHY-101	Physical Principles	Physics	4

course (course_id, title, dept_name, credits)

section

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

section (course_id, sec_id, semester, year, building, room_number, time_slot_id)

teaches

id	course_id	sec_id	semester	year
10101	CS-101	1	Fall	2009
10101	CS-315	1	Spring	2010
10101	CS-347	1	Fall	2009
12121	FIN-201	1	Spring	2010
15151	MU-199	1	Spring	2010
22222	PHY-101	1	Fall	2009
76766	BIO-101	1	Summer	2009
76766	BIO-301	1	Summer	2010
83821	CS-190	1	Spring	2009
83821	CS-190	2	Spring	2009
83821	CS-319	2	Spring	2010
98345	EE-181	1	Spring	2009

teaches (ID, course_id, sec_id, semester, year)

teacher

id	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

teacher (ID, name, dept_name, salary)

department

dept_name	building	budget
Biology	Watson	90000.00
Comp. Sci.	Taylor	100000.00
Elec. Eng.	Taylor	85000.00
Physics	Watson	70000.00

department (dept_name, building, budget)

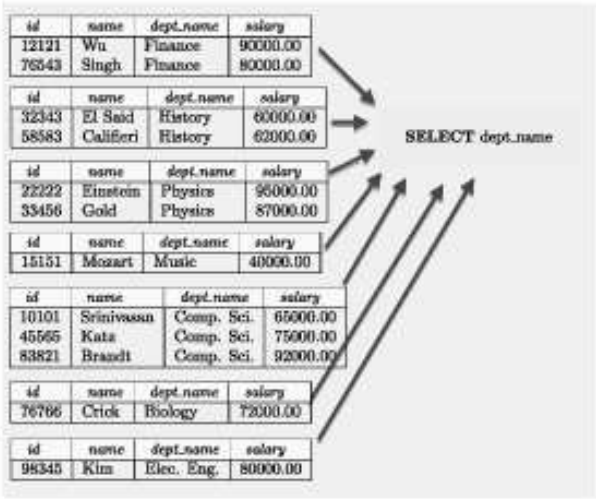
Group By

```
SELECT dept_name
FROM teacher
GROUP BY dept_name;
```

dept_name
Finance
History
Physics
Music
Comp. Sci.
Biology
Elec. Eng.

teacher

id	name	dept_name	salary
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00



GROUP BY:

Step 1. split input table into mini-tables by dept_name

Step 2. run query on each table separately to get *one* row per mini-table

Group By

```
myuniversitydb=# SELECT dept_name
myuniversitydb-# FROM teacher
myuniversitydb-# GROUP BY dept_name;
 dept_name
------------
 Finance
 History
 Physics
 Music
 Comp. Sci.
 Biology
 Elec. Eng.
(7 rows)
```

Group By

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the average salary in each department

```
SELECT dept_name, avg(salary)
FROM teacher
GROUP BY dept_name;
```

How to understand this query ?

Group By

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT dept_name, avg(salary)
FROM teacher
GROUP BY dept_name;
```

Step 1: a separate table for all rows with the same value on dept_name

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
45565	Katz	Comp. Sci.	75000.00
83821	Brandt	Comp. Sci.	92000.00

id	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000.00
33456	Gold	Physics	87000.00

id	*name*	*dept_name*	*salary*
76766	Crick	Biology	72000.00

id	*name*	*dept_name*	*salary*
98345	Kim	Elec. Eng.	80000.00

id	*name*	*dept_name*	*salary*
12121	Wu	Finance	90000.00
76543	Singh	Finance	80000.00

id	*name*	*dept_name*	*salary*
32343	El Said	History	60000.00
58583	Califieri	History	62000.00

id	*name*	*dept_name*	*salary*
15151	Mozart	Music	40000.00

Step 2: run the query separately on each table to get a single row

```
SELECT dept_name, avg(salary)
```

dept_name	*avg*
Comp. Sci.	77333.333333333333

dept_name	*avg*
Physics	91000.000000000000

dept_name	*avg*
Biology	72000.000000000000

dept_name	*avg*
Elec. Eng.	80000.000000000000

dept_name	*avg*
Finance	85000.000000000000

dept_name	*avg*
History	61000.000000000000

dept_name	*avg*
Music	40000.000000000000

Group By

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT dept_name, avg(salary)
FROM teacher
GROUP BY dept_name;
```

Step 2: run the query separately on each table to get a single row

```
SELECT dept_name, avg(salary)
```

dept_name	*avg*
Comp. Sci.	77333.333333333333

dept_name	*avg*
Physics	91000.000000000000

dept_name	*avg*
Biology	72000.000000000000

dept_name	*avg*
Elec. Eng.	80000.000000000000

dept_name	*avg*
Finance	85000.000000000000

dept_name	*avg*
History	61000.000000000000

dept_name	*avg*
Music	40000.000000000000

Step 3: put the single row from each table together to get result

dept_name	*avg*
Finance	85000.000000000000
History	61000.000000000000
Physics	91000.000000000000
Music	40000.000000000000
Comp. Sci.	77333.333333333333
Biology	72000.000000000000
Elec. Eng.	80000.000000000000


```
SELECT dept_name, avg(salary)
FROM teacher
GROUP BY dept_name;
```


we get a single row for each mini-table because:

1. dept_name is the **same** for all rows in each mini-table, and
2. **avg**(salary) is a single number for each mini-table

$\Longrightarrow$ any attribute in **SELECT** must appear in **GROUP BY**:

$\longrightarrow$ so that all attribute values are the same in each mini-table
$\longrightarrow$ and so this table can be merged to a single row

Group By

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the average salary in each department

```
SELECT dept_name, avg(salary)
FROM teacher
GROUP BY dept_name;
```

```
myuniversitydb=# SELECT dept name, avg(salary)
myuniversitydb-# FROM teacher
myuniversitydb-# GROUP BY dept_name;
 dept_name  |        avg
------------+--------------------
 Finance    | 85000.000000000000
 History    | 61000.000000000000
 Physics    | 91000.000000000000
 Music      | 40000.000000000000
 Comp. Sci. | 77333.333333333333
 Biology    | 72000.000000000000
 Elec. Eng. | 80000.000000000000
(7 rows)
```

dept_name	*avg*
Finance	85000.000000000000
History	61000.000000000000
Physics	91000.000000000000
Music	40000.000000000000
Comp. Sci.	77333.333333333333
Biology	72000.000000000000
Elec. Eng.	80000.000000000000

Group By

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

Find the number of teachers in each department.

```
SELECT dept_name, count(*)
FROM teacher
GROUP BY dept_name;
```

```
myuniversitydb=# SELECT dept name, count(*)
myuniversitydb-# FROM teacher
myuniversitydb-# GROUP BY dept name;
 dept name  | count
------------+-------
 Finance    |     2
 History    |     2
 Physics    |     2
 Music      |     1
 Comp. Sci. |     3
 Biology    |     1
 Elec. Eng. |     1
(7 rows)
```

dept_name	*count*
Finance	2
History	2
Physics	2
Music	1
Comp. Sci.	3
Biology	1
Elec. Eng.	1

Group By

section

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

```
SELECT semester, year
FROM section
GROUP BY sec_id, semester, year;
```

Always:

Understand query by looking at it step by step

Group By

section

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

```
SELECT semester, year
FROM section
GROUP BY sec_id, semester, year;
```

GROUP BY sec_id, semester, year

Step 1: a separate table for all rows with the same value on sec_id, semester, year

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
CS-190	2	Spring	2009	Taylor	3128	A

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-301	1	Summer	2010	Painter	514	A

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
CS-101	1	Spring	2010	Packard	101	F
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
FIN-201	1	Spring	2010	Packard	101	B
MU-199	1	Spring	2010	Packard	101	D
HIS-351	1	Spring	2010	Painter	514	C

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
CS-101	1	Fall	2009	Packard	101	H
CS-347	1	Fall	2009	Taylor	3128	A
PHY-101	1	Fall	2009	Watson	100	A

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
CS-190	1	Spring	2009	Taylor	3128	E
EE-181	1	Spring	2009	Taylor	3128	C

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
CS-319	2	Spring	2010	Taylor	3128	C

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B

Step 2: run the query separately on each table to get a single row per table

```
SELECT semester, year
```

semester	*year*
Spring	2009

semester	*year*
Spring	2009

semester	*year*
Fall	2009

semester	*year*
Summer	2010

semester	*year*
Summer	2009

semester	*year*
Spring	2010

semester	*year*
Spring	2010

Group By

```
SELECT semester, year
FROM section
GROUP BY sec_id, semester, year;
```

GROUP BY sec_id, semester, year

section

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

Step 2: run the query separately on each table to get a single row per table

```
SELECT semester, year
```

semester	year
Spring	2009

semester	year
Spring	2009

semester	year
Fall	2009

semester	year
Summer	2010

semester	year
Summer	2009

semester	year
Spring	2010

semester	year
Spring	2010

Step 3: put the single row from each table together to get result

semester	year
Summer	2010
Spring	2010
Fall	2009
Spring	2009
Spring	2010
Spring	2009
Summer	2009

- different mini-tables can give same result
- we keep duplicates.

Group By

section

course_id	sec_id	semester	year	building	room_number	time_slot_id
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

```
SELECT distinct semester, year
FROM section
GROUP BY sec_id, semester, year;
```

```
myuniversitydb=# SELECT distinct semester, year
myuniversitydb-# FROM section
myuniversitydb-# GROUP BY sec_id, semester, year;
 semester | year
----------+------
 Fall     | 2009
 Spring   | 2010
 Summer   | 2010
 Summer   | 2009
 Spring   | 2009
(5 rows)
```

GROUP BY: LOGICAL ORDER

SELECT ···
FROM ···
WHERE ···
GROUP BY ···

① **FROM** table1, table2, ···

compute the input table from table1 × table2 × ···

② **WHERE** condition

choose rows in the input table that satisfy condition

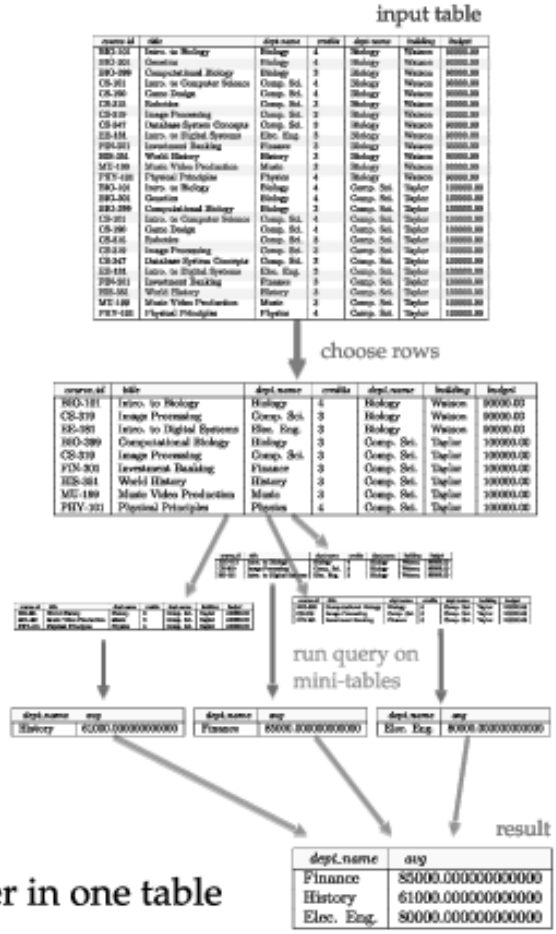
③ **GROUP BY** ···

break rows into mini-tables, with each mini-table having same values for the **GROUP BY** attributes

④ **SELECT** ···

run **SELECTION** statement *separately* on each mini table to get one output row per mini-table

the result is all the output rows put together in one table



dept_name	avg
Finance	85000.00000000000
History	61000.00000000000
Elec. Eng.	80000.00000000000

Important:

run **SELECTION** statement *separately* on each mini table to get **one output row** per mini-table

④ **SELECT** ···

run **SELECTION** statement *separately* on each mini table to get one output row per mini-table

Group By

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT dept_name, id, avg(salary)
FROM teacher
GROUP BY dept_name;
```

Understand query by looking at it step by step

Step 1: a separate table for all rows with the same value on dept_name

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
45565	Katz	Comp. Sci.	75000.00
83821	Brandt	Comp. Sci.	92000.00

id	*name*	*dept_name*	*salary*
12121	Wu	Finance	90000.00
76543	Singh	Finance	80000.00

id	*name*	*dept_name*	*salary*
22222	Einstein	Physics	95000.00
33456	Gold	Physics	87000.00

id	*name*	*dept_name*	*salary*
32343	El Said	History	60000.00
58583	Califieri	History	62000.00

id	*name*	*dept_name*	*salary*
76766	Crick	Biology	72000.00

id	*name*	*dept_name*	*salary*
98345	Kim	Elec. Eng.	80000.00

id	*name*	*dept_name*	*salary*
15151	Mozart	Music	40000.00

Step 2: compute a *single* row for each table by applying

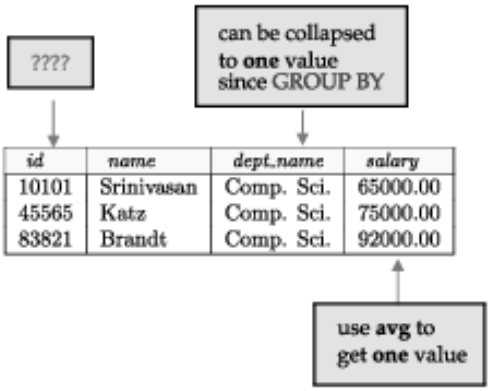
```
SELECT dept_name, id, avg(salary)
```

separately to each table

Group By

```
SELECT dept_name, id, avg(salary)
FROM teacher
GROUP BY dept_name;
```

Understand query by looking at it step by step



id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
45565	Katz	Comp. Sci.	75000.00
83821	Brandt	Comp. Sci.	92000.00

Step 1: a separate table for all rows with the same value on dept_name

Step 2: compute a *single* row for each table by applying

```
SELECT dept_name, id, avg(salary)
```

separately to each table

```
SELECT dept_name, id, avg(salary)
FROM teacher
GROUP BY dept_name;
```



```
myuniversitydb=# SELECT dept name, id, avg(salary)
myuniversitydb-# FROM teacher
myuniversitydb # GROUP BY dept name;
ERROR:  column "teacher.id" must appear in the GROUP BY clause or be used in an aggregate functio
LINE 1: SELECT dept name, id, avg(salary)
                          ^
```

Group By

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00




```
SELECT dept_name, salary, count(*)
FROM teacher
GROUP BY dept_name;
```



```
myuniversitydb-# SELECT dept name, salary, count(*)
myuniversitydb-# FROM teacher
myuniversitydb-# GROUP BY dept_name;
ERROR:  column "teacher.salary" must appear in the GROUP BY clause or be used in an aggregate function
LINE 1: SELECT dept name, salary, count(*)
                          ^
```

Recall: any attribute in **SELECT** must appear in **GROUP BY**

Group By

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

any attribute in SELECT must appear in GROUP BY

the other direction is not **required** :

if in GROUP BY, does not **have** to appear in SELECT

```
myuniversitydb=# SELECT count(*)
myuniversitydb-# FROM teacher
myuniversitydb-# GROUP BY dept name;
 count
-------
     2
     2
     2
     1
     3
     1
     1
(7 rows)
```

```
SELECT count(*)
FROM teacher
GROUP BY dept_name;
```

count
2
2
2
1
3
1
1

Group By

section

course_id	*sec_id*	*semester*	*year*	*building*	*room_number*	*time_slot_id*
BIO-101	1	Summer	2009	Painter	514	B
BIO-301	1	Summer	2010	Painter	514	A
CS-101	1	Fall	2009	Packard	101	H
CS-101	1	Spring	2010	Packard	101	F
CS-190	1	Spring	2009	Taylor	3128	E
CS-190	2	Spring	2009	Taylor	3128	A
CS-315	1	Spring	2010	Watson	120	D
CS-319	1	Spring	2010	Watson	100	B
CS-319	2	Spring	2010	Taylor	3128	C
CS-347	1	Fall	2009	Taylor	3128	A
EE-181	1	Spring	2009	Taylor	3128	C
FIN-201	1	Spring	2010	Packard	101	B
HIS-351	1	Spring	2010	Painter	514	C
MU-199	1	Spring	2010	Packard	101	D
PHY-101	1	Fall	2009	Watson	100	A

```
SELECT semester, year
FROM section
GROUP BY sec_id, semester, year;
```

```
myuniversitydb # SELECT semester, year
myuniversitydb # FROM section
myuniversitydb # GROUP BY sec_id, semester, year;
 semester | year
----------+------
 Summer   | 2010
 Spring   | 2010
 Fall     | 2009
 Spring   | 2009
 Spring   | 2010
 Spring   | 2009
 Summer   | 2009
(7 rows)
```

```
SELECT semester, year
FROM section
WHERE semester = 'Spring'
GROUP BY sec_id, year;
```

```
myuniversitydb=# SELECT semester, year
                 FROM section
                 WHERE semester = 'Spring'
                 GROUP BY sec_id, year;
ERROR:  column "section.semester" must appear in the
 GROUP BY clause or be used in an aggregate function
LINE 1: SELECT semester, year
                ^
```

SQL requires certain rules, even if not necessary logically!

Group By

teacher

id	*name*	*dept_name*	*salary*
10101	Srinivasan	Comp. Sci.	65000.00
12121	Wu	Finance	90000.00
15151	Mozart	Music	40000.00
22222	Einstein	Physics	95000.00
32343	El Said	History	60000.00
33456	Gold	Physics	87000.00
45565	Katz	Comp. Sci.	75000.00
58583	Califieri	History	62000.00
76543	Singh	Finance	80000.00
76766	Crick	Biology	72000.00
83821	Brandt	Comp. Sci.	92000.00
98345	Kim	Elec. Eng.	80000.00

```
SELECT dept_name, max(salary), count(*)
FROM teacher
GROUP BY dept_name;
```



dept_name	*max*	*count*
Finance	90000.00	2
History	62000.00	2
Physics	95000.00	2
Music	40000.00	1
Comp. Sci.	92000.00	3
Biology	72000.00	1
Elec. Eng.	80000.00	1

Logical Order of Processing

FROM ⟶ WHERE ⟶ SELECT



DISTINCT ?

FROM ⟶ WHERE ⟶ SELECT ⟶ DISTINCT



AGGREGATIONS ?

FROM ⟶ WHERE ⟶ AGGREGATIONS ⟶ SELECT ⟶ DISTINCT



GROUP BY ?

FROM ⟶ WHERE ⟶ GROUP BY ⟶ AGGREGATIONS ⟶ SELECT ⟶ DISTINCT

⟶ aggregations available from here onwards
applied to each mini-table separately